Lab Report: Introduction to ROS 2 via Turtlesim

Course: MCT-454L Mobile Robotics

Date: March 4, 2026

Submitted to: Dr. Maria Akram

Submitted By: Syeda Imaan Fatima 2022-MC-28

1. Objective

The primary goal of this lab was to transition from theoretical robotics concepts to practical implementation using **ROS 2 Humble**. By utilizing the turtlesim package, we aimed to:

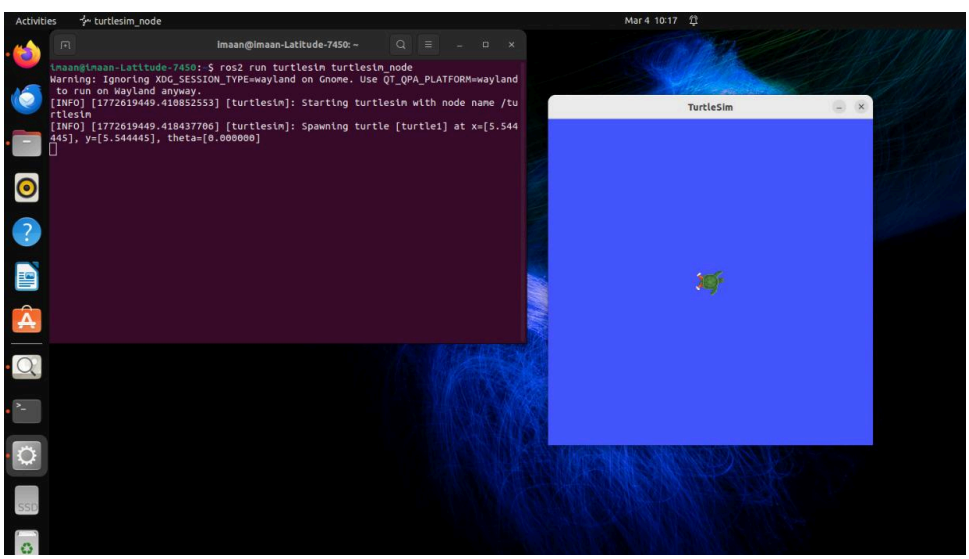
- Understand the execution and management of **Nodes**.
- Observe real-time data flow through **Topics** (Publishers and Subscribers).
- Utilize **CLI tools** (ros2 topic, ros2 service) and graphical tools (**RQT**) to monitor and manipulate a robotic system.

2. Methodology & Procedures

Phase 1: Environment Initialization and Node Execution

The lab began by sourcing the ROS 2 environment and launching the core simulator.

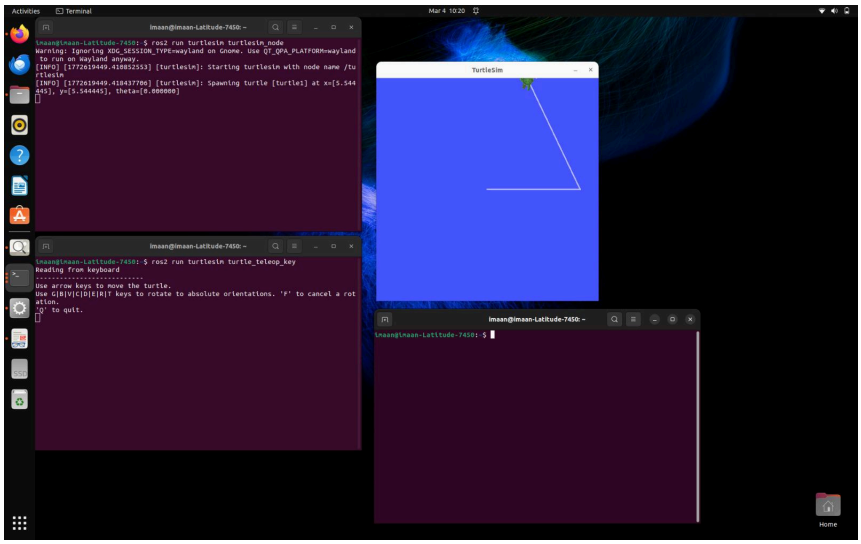
- **Command:** sudo apt install turtlesim
- **Command:** ros2 run turtlesim turtlesim_node
- **Observation:** A blue simulation window appeared with a single turtle centered at coordinates approximately x=5.5, y=5.5. The terminal provided metadata regarding the spawning of [turtle1].

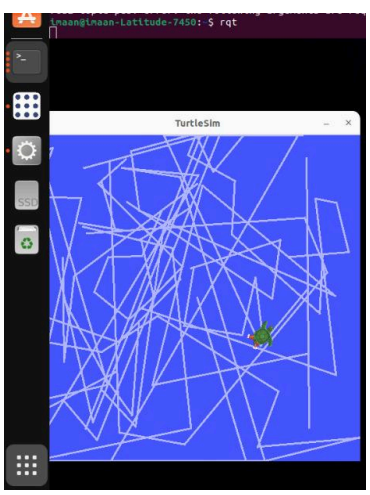


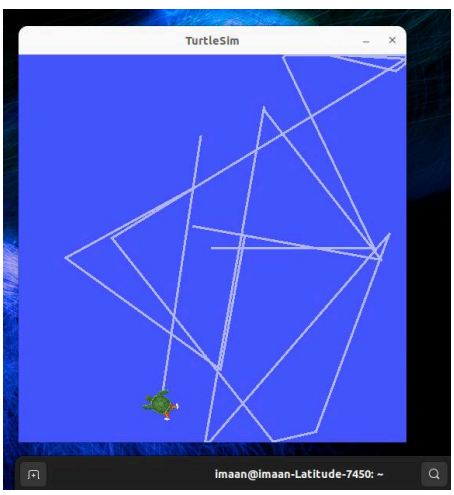
Phase 2: Teleoperation and Message Monitoring

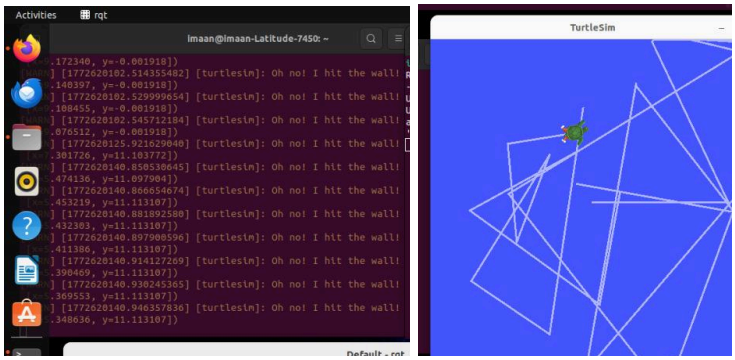
To move the turtle, a second node was launched in a separate terminal to capture keyboard inputs.

- **Command:** ros2 run turtlesim turtle_teleop_key
- **Observation:** Using arrow keys, the turtle moved within the GUI. When the turtle reached the boundary of the simulator, the turtlesim_node terminal displayed [WARN] messages: *"Oh no! I hit the wall! (Clamping from...)"*. This demonstrated how nodes communicate status and constraints back to the user.





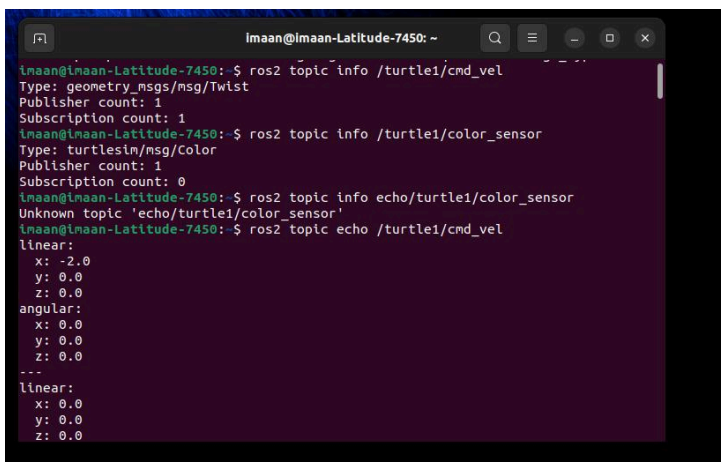


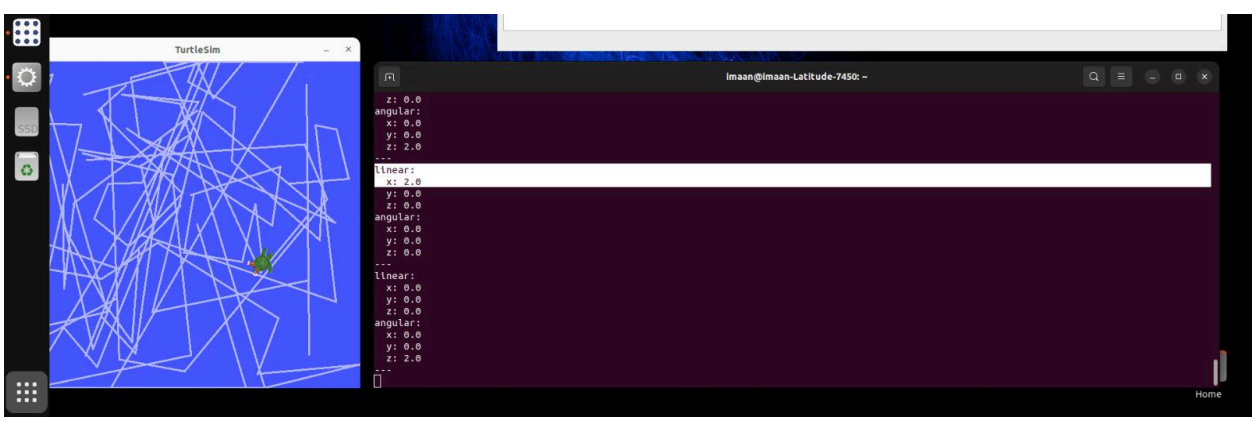


Phase 3: Inspecting the "Plumbing" (Topics & Messages)

To understand how the keyboard "talks" to the turtle, we inspected the active communication channels.

- **Topic List:** Running ros2 topic list revealed /turtle1/cmd_vel (Command Velocity) and /turtle1/pose (Position/Orientation).
- **Data Echoing:** By running ros2 topic echo /turtle1/cmd_vel, we observed the raw Twist messages.
    - **Idle State:** The message showed 0.0 for all linear and angular fields.
    - **Active State:** Pressing the 'Up' arrow changed the linear: x value (e.g., 2.0), while the 'Left/Right' arrows changed the angular: z value.



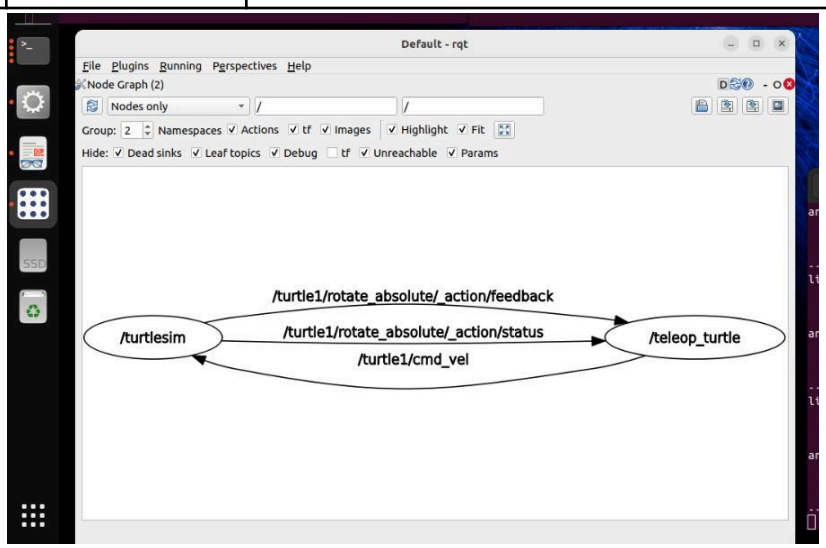


3. Graphical Analysis via RQT

3.1 Graphical Node Graph (rqt_graph)

The **RQT Node Graph** was used to visualize the system architecture. As shown in the captured screenshots, the graph clearly defines the relationship between the components:

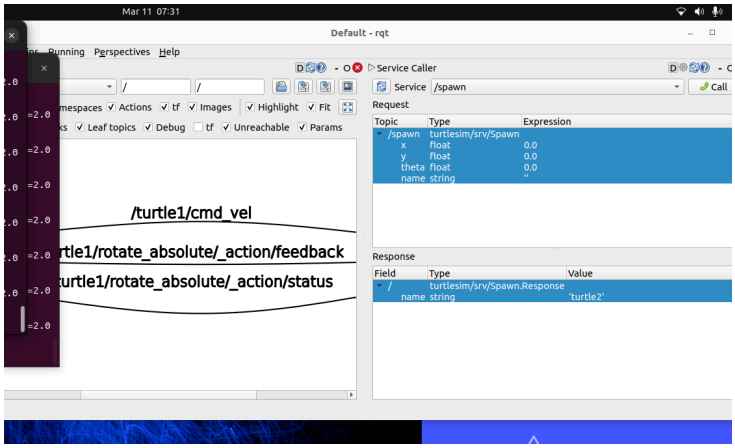
Component	Role	Description
/teleop_turtle	Publisher	Captures keystrokes and sends velocity data to /turtle1/cmd_vel.
/turtlesim	Subscriber	Receives movement commands and updates the GUI.
/turtle1/cmd_vel	Topic (Twist)	The main stream for continuous linear and angular velocity.



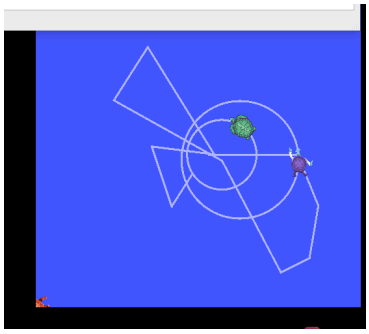
3.2 Service Caller: Spawning and Resetting

A key part of the exercise was interacting with ROS 2 **Services** via the RQT GUI.

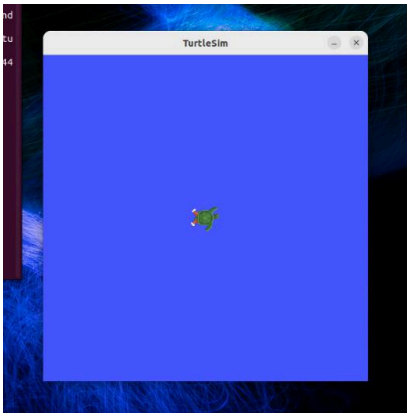
- **The `/spawn` Service:** We navigated to the Service Caller and provided x, y, and name parameters.

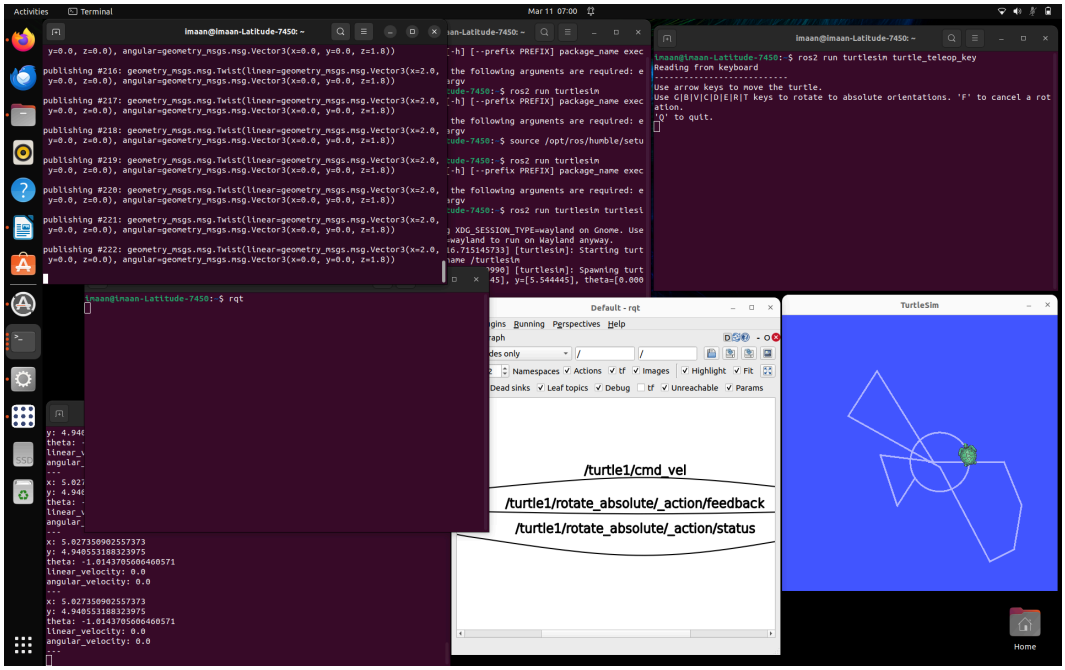


- **Result:** As confirmed in the latest screenshots, a second turtle (`turtle2`) appeared in the simulator.



- **The `/reset` Service:** Calling this service instantly cleared the path history (the white lines) and repositioned the turtles, demonstrating the "Request-Response" nature of services.





3.3 Manual Publishing & Troubleshooting

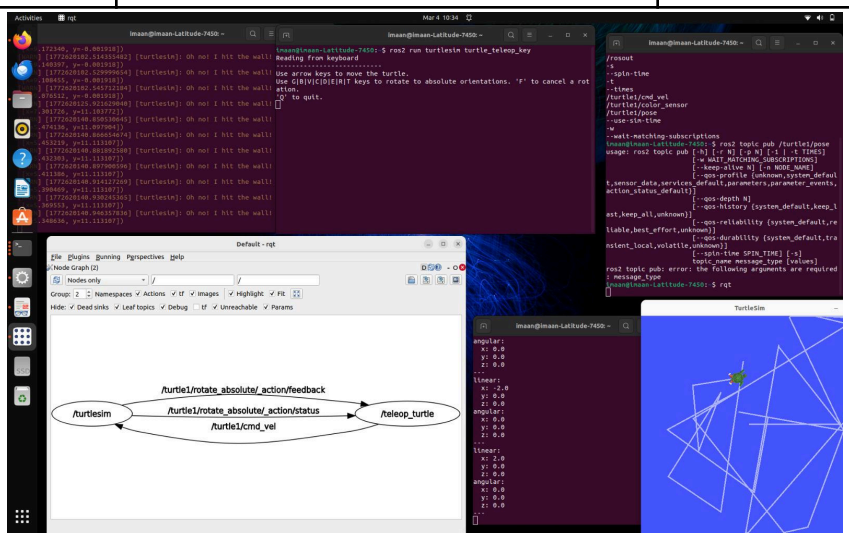
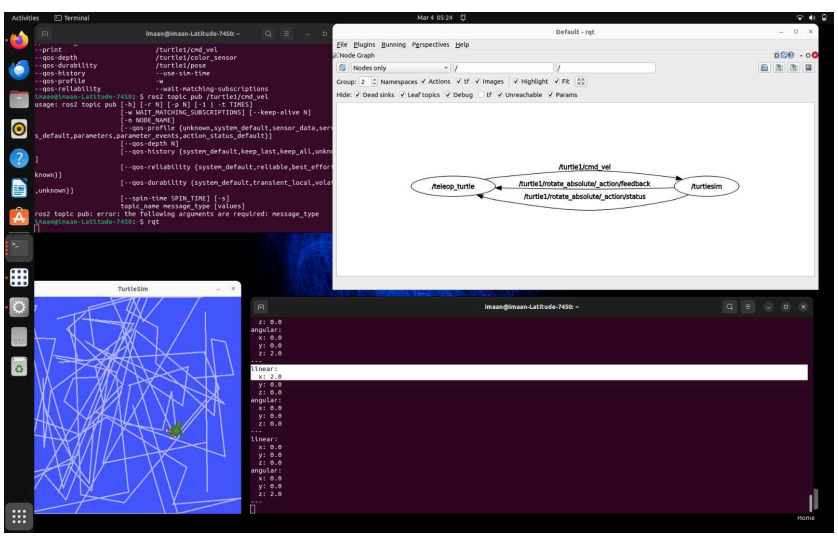
We tested the `ros2 topic pub` command to drive the turtle directly from the terminal.

- **Observation:** By sending a `geometry_msgs/msg/Twist` message with a linear x of 2.0 and an angular z of 1.8, we achieved a steady circular motion.
- **Learning Point:** This highlighted that any node (or the user) can publish to a topic as long as the message type and syntax are correct.

4. Conclusion & Observations

This lab successfully demonstrated the modular nature of ROS 2. We observed that:

1. **Nodes are Independent:** The simulator doesn't care *how* it receives data, only that the data arrives on the correct topic (/cmd_vel).
2. **Transparency:** Using echo and rqt_graph, we can peak into the "brain" of the robot to debug exactly what signals are being sent.
3. **Scalability:** The ability to spawn multiple turtles and control them via different topics highlights the flexibility of the ROS 2 framework for complex multi-robot systems.
4. **Clamping:** The terminal warned us when the turtle hit a wall, showing that the `turtlesim_node` has built-in safety logic.
5. **Feedback Loops:** The RQT graph showed `/action/feedback` lines, indicating that the robot reports back its progress during complex maneuvers.

6. **Strict Syntax:** The failed manual publish attempt taught us that ROS 2 requires precise message definitions to ensure data integrity.

Feature	Topic (/turtle1/pose)	Service (/spawn)
Data Flow	Continuous/Asynchronous	Discrete/Synchronous
Logic	"Here is my current position."	"Create a new robot at these coordinates."
Efficiency	High frequency for real-time sensing.	Low frequency for configuration.